UU_Inference Write Up

By: Sreekaanth, Srinivasan, Lior Belay, Paola Quiroga, Keshav Bhakta, Savannah Fox

Our goal is to set the price of our goods so that it maximizes revenue without sampling the population unrealistically. This requires balancing the tradeoff between the number of customers and the revenue earned per customer. When prices are low we attract a large number of customers, but each customer contributes a smaller revenue. As we raise prices the number of customers falls however revenue per customer increases.

We followed the approach recommended by the professor, beginning with a grid search from 0 to 70 to find the general curve of the predicted revenue and identify local maxima. We used steps of two, which gave 35 sample price points on the curve. In step two, we narrowed our search to two local maxima from the three identified in the grid search. In step three, we performed a gradient ascent on the two local maxima to identify their peak values which will be the predicted optimized price. We chose this approach because the search for local maximum needs minimal test subjects and leads us to a starting point of where to look for the actual optimized price that is unknown to us. Gradient ascent then refines our estimate of each of the peaks to identify the optimal price.

To account for the randomness in individual purchase decisions, we quantified uncertainty at every stage rather than relying on single revenue numbers. During each gradient update we calculated the standard error of the revenue differences between the lower and the upper test price, this allowed us to know how much to trust each signal before moving. After gradient ascent converged from both starting points, we did not just pick the finalist with the higher revenue. Instead we tested each finalist price on 3,000 new customers and ran a two sample z-test to compare them. We only called one price better if the 95% confidence interval for the difference did not include zero. This kept up from picking a price just because of the random noise in the data.

Our biggest challenge was that the revenue has multiple peaks and not just one. This meant that starting gradient ascent from a single price could get stuck at a smaller peak instead of finding the best price overall. We fixed this by first running a grid search across the full $0 - $70 range to see the shape of the curve and find all the candidate peaks. We then picked the two most separated peaks as starting points for gradient ascent. A second challenge was that our gradient estimates were noisy with only small samples, which sometimes pushed the price in the wrong direction. We fixed this by shrinking the learning rate over time and capping how far the price could move in one step. This stopped early noisy updates from overshooting while still letting the price move quickly once our estimates got more reliable.

Another challenge that we faced was that the grid search was also noisy. Since we only had about 100 customers per price, the revenue estimates that had higher prices had large standard errors, because each sale is worth more and the revenue depends on the customer. This makes the $30 grid point look like the best price even though the actual value is closer to $10, and the actual peak sits between the 26-28 dollar grid points. We solved this by using the grid only to find regions and by letting gradient ascent and the final z-test make the decisions.

Our chosen price was $27.71. The grid search gave us two starting points of $30 and $12. Gradient ascent moved the price down from $30 and then fluctuated around $27.7, since each gradient estimate was noisy. The gradient ascent from $12 ended at $12.25. We then tested them on 3,000 new customers each and $27.71 earned about $12.03 per customer while $12.25 earned about $11.50, and the 95% confidence interval for the difference excluded zero, so we chose $27.71. We experimented on a total of 33,392 customers and earned $361,239 in revenue, which averaged to about $10.82 per customer. Since the optimal price is $11.925 per customer, we have a remainder of 9.3% in cost of learning because the customers were testing at poor prices during the grid search and on the lower peaks which earned less than the optimal price.